Step 1: Strict Architectural Isolation (The Control) To eliminate the architectural biases present in past research, you must select your Small Language Model (SLM) and Large Language Model (LLM) baselines from the **exact same model family**. Utilizing the Qwen2.5 family (e.g., Qwen2.5-1.5B or 3B as the SLM and Qwen2.5-VL-7B or 14B as the LLM) guarantees that both models share an **identical vision encoder, tokenizer, and vocabulary**. By decoupling the visual perception mechanics from the textual reasoning backbone, you ensure that any performance deltas are scientifically attributed strictly to the parameter scale of the reasoning engine, rather than a mismatched vision module.

Step 2: Unified Data-Ingestion Pipeline Engineering (The Pipeline) Standard evaluations often fail because they allow different models to use their own default, black-box preprocessing scripts, leading to inherent data bias. To prevent this, you must **bypass the default preprocessing scripts** entirely. Instead, build a custom, identical frame-sampling pipeline using optimized computer vision libraries such as OpenCV and FFmpeg. This guarantees that both the SLM and the LLM are fed the **exact same array of pixels** and visual tokens, leveling the sensory playing field before reasoning begins.

Step 3: High-Fidelity Data Curation and Semantic Filtering Instead of using noisy, uncurated web-scraped video data, implement a **semantic-based data filtering strategy** to mitigate content homogeneity and ensure diverse, high-density training/testing data. Utilize a dense Vision-Language Model (VLM) or LLM as a semi-automatic annotator to filter out irrelevant or redundant visual frames, retaining only the most semantically rich inputs. This step ensures the SLM is evaluated on pristine, mathematically normalized data (such as discrete temporal tokens) rather than raw, noisy timecodes that inherently disadvantage smaller models.

Step 4: Dynamic Visual Compression and Frame Selection To avoid the perception bottlenecks of standard uniform frame sampling—which blindly skips important visual cues—you must apply **semantic-aware key-frame extraction** (dynamic compression). This involves extracting global features from the video and calculating inter-frame similarities to discard redundant frames. Additionally, prune uninformative spatial tokens (e.g., static background patches) and merge semantically similar tokens. This step ensures the models are tested on their ability to reason over dense, salient information rather than their ability to memorize empty spatial noise.

Step 5: Inference Compute (FLOP) Standardization A major bias in past VLM scaling research is allowing larger models to process massive amounts of visual tokens without accounting for the exponential computational cost. To make the experiment scientifically fair, you must **fix the inference compute budget**. Because inference FLOPs are dictated by the LLM parameter size (N) multiplied by the visual token count (T), you must enforce strict trade-offs. If you test the 14B LLM, you must aggressively compress its visual inputs (e.g., down to 1-4 tokens per frame), and compare it against the 1.5B SLM processing denser tokens under the exact same computational ceiling.

Step 6: Optimized Orchestration and Edge-Deployment Testing Inference loops must be highly engineered to **maximize parallel processing**. If orchestration bottlenecks occur during the data transfer phase, hardware latency will artificially skew the model evaluation metrics. Deploy the models on physical edge hardware to track the disparity between the compute-bound prefill stage (reading the prompt and video) and the memory-bound decode stage (generating the answer).

Step 7: Holistic Matrix Evaluation Finally, evaluate the models strictly through a dual-faceted matrix that measures both cognitive intelligence and physical systems-level efficiency.

- **Task Accuracy:** Test the models against standardized benchmarks (like Video-MME) using exact-match or LLM-judged criteria.
- **Computational Cost:** Log the **Time to First Token (TTFT), Tokens Per Second (TPS), and Peak VRAM utilization** to definitively prove whether the data-engineered SLM achieves Pareto-optimal efficiency over the LLM.